# 23CSE455 Design Patterns

## Case Study Report

**Team:** A1

**Case study:** Food delivery application built with Django: discovery, evolution and cross-language comparison of the design patterns in a working campus food-ordering system.

**Application:** HarshavardhanVemali/campus-food-ordering ("Canteen Management System", MIT licence). Approved by the course faculty as Team A1's application.

**Date:** 2026-10-11

**Submission repository:** 23CSE455_DP_CaseStudy, folder `A1/`

**Students**

| Roll number | Student name | Batch |
|---|---|---|
| AM.SC.U4CSE23001 | Aadil Nizar | CSE A |
| AM.SC.U4CSE23007 | Adil Najeem | CSE A |
| AM.SC.U4CSE23014 | Athul Yesudas | CSE A |
| AM.SC.U4CSE23039 | Hima C | CSE A |
| AM.SC.U4CSE23046 | M Krishnapriya | CSE A |
| AM.SC.U4CSE23053 | Muhammed Faais A | CSE A |

# 1 Application and architecture

**Original source:** Repository `https://github.com/HarshavardhanVemali/campus-food-ordering` (`main`). There is no release tag, so the snapshot is identified by commit `83998629e633594037cfa25e583542f9f6f0f968`. The commit archive (SHA-256 `104b48ea...0263c`, `data/provenance/provenance.json`) contains 170 files, and each file's hash is listed in `data/provenance/baseline-files.sha256.json`. The copy in `data/code/baseline_83998629/` keeps 167 files byte-identical. Two files carry one-line credential redactions (`data/provenance/REDACTIONS.md`), and a 15 MB stock video is left out of the package; `data/code/verify_baseline.py` checks all three groups. The team's earlier download (`campus-food-ordering-main.zip`) has the same content, and a repository screenshot from 2026-09-12 (`data/provenance/github_repository_page_2026-09-12.png`) shows the same short hash, `8399862`. The repository page was captured on 2026-09-12 and the commit archive was fetched on 2026-10-11. Licence: MIT (`LICENSE`, copyright 2025 Harshavardhan Vemali).

**Environment:** Python with Django and Django Channels (ASGI through Daphne); SQLite (`settings.DATABASES`); the Channels `InMemoryChannelLayer` (`canteen/settings.py:131-135`); django-allauth and social-auth for Google sign-in; Firebase Admin for push messages; and the PhonePe and PayU payment SDKs. `requirements.txt` pins 99 packages, among them Django 6.0.4, channels 4.3.2, daphne 4.2.1, django-allauth 65.16.1, firebase-admin 7.4.0 and phonepe-sdk 1.1.0. The upstream README mentions Django 5.1 and Redis, but the pinned requirements and `settings.py` do not use them, and this report follows the code. CH01 was validated with Python 3.12.14, Django 6.0.4, Channels 4.3.2 and SQLite 3.53.1 (`Part2/changes/Change1/evidence/runtime-versions.txt`; full package list in `environment-freeze.txt`). Part 3 used OpenJDK 21.0.12.1, CPython 3.13.16, Node.js 22.22.0 and g++ 13.3.0 (`Part3/evidence/toolchain_versions.txt`).

**Scope:** The application consists of one Django app, `canteenapp`, and the project package `canteen`. Inspected in detail: `canteenapp/models.py` (629 lines), `views.py` (5,363 lines), `consumers.py`

(946 lines), `signals.py`, `apps.py`, `routing.py`, `canteen/settings.py`, `urls.py`, `asgi.py` and the template `adminassignorder.html`. A reproducible scan (`Part1/evidence/scan_candidates.sh`) covered every Python module of `canteenapp`. Participants were verified in three areas that the case study and its changes depend on: item availability, the order-status lifecycle and delivery assignment. Excluded: the other 62 templates and the static CSS and JavaScript, which hold presentation only; the 17 migration files, which are generated schema history; and the internals of Django, Channels and social-auth, whose roles are taken from their documentation [3, 4]. The payment and notification code was read only as far as needed to judge the Facade, Adapter and Singleton candidates.

**Architecture:** Figure 1 shows the components. `canteen/asgi.py` sends plain HTTP to Django and passes WebSocket connections through `AuthMiddlewareStack` to the consumers listed in `canteenapp/routing.py`. Most business logic lives in function-based views, although models such as `Order` also calculate prices and handle stock. An order-status update shows how the parts work together:

1. an administrator's page posts JSON to `/adminupdateorderstatus/`;
2. the view checks that the status does not move backwards (`get_status_index`) and saves the `Order`;
3. it calls `group_send` on the channel layer for the admin, restaurant and delivery groups;
4. Channels delivers each event to every consumer in the group and calls the handler named by the event type;
5. each handler re-reads the relevant orders and pushes JSON over the WebSocket to its browser.

A second, synchronous path runs inside the ORM: saving an `Item` emits `post_save`, and a receiver deletes cart rows for unavailable items.

**Run instructions:** The upstream setup steps are in `data/code/baseline_83998629/README.md`: create a virtual environment, run `pip install -r requirements.txt`, create `.env` from `.env.example`, apply migrations and start the ASGI server. Followed literally, the installation fails: `phonepe-sdk` is not on the default package index, `psycopg2` needs `pg_config`, and several pinned packages conflict (`Part2/changes/Change1/evidence/install*.log`). A validation environment was therefore assembled as recorded in `Part2/changes/Change1/verification/commands.md`. In that environment `python manage.py check` exits 0 with one deprecation warning (`ACCOUNT_EMAIL_REQUIRED`) and a message that the Firebase service-account file is missing, and `python manage.py makemigrations --check --dry-run` reports no changes. The development server was not started for this study.

## 2 Part 1 Pattern discovery

**Method:** Candidates came from three sources: the team's earlier analysis note (`Part1/llm/historical_note_pattern_analysis.md`), a reading of the modules in scope, and a scripted scan of the baseline (`Part1/evidence/candidate_scan.txt`). The scan looked for signal receivers, Channels subscriptions and publications, routed consumers, abstract classes, class names suggesting GoF roles, status tables, module-level singletons, the social-auth pipeline and access-control decorators. A candidate was confirmed only when three conditions held:

1. every role of the pattern mapped to application code or to a documented framework mechanism;
2. the calls forming the collaboration could be traced in both directions;
3. at least one concrete participant varied independently of the code that triggers it.

A class name, an inheritance link or an LLM statement was not accepted on its own. Line ranges come from Python's `ast` module and include decorators.

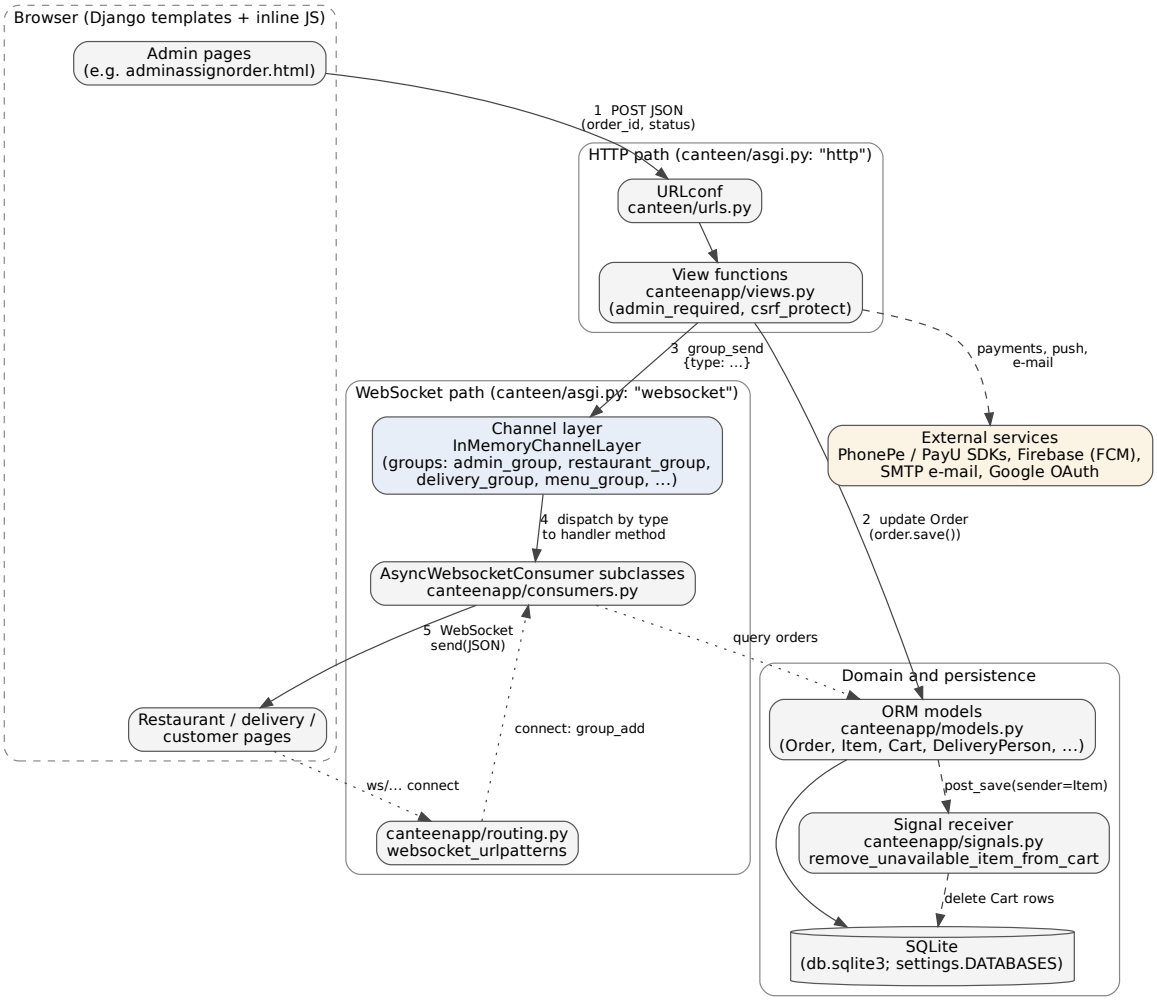



**Figure 1:** Components and the order-status request/event flow at commit 83998629. Source: `Part1/pattern_architecture/architecture.dot` (Graphviz), with the rendered PDF and PNG beside it.

### 2.1 Totals at the baseline

**Confirmed pattern types:** 1: Observer.

**Confirmed pattern instances:** 2: Item Unavailability Cart Cleanup and Order Status Broadcast.

**Concrete participant classes:** 3: `AdminOrderConsumer`, `RestaurantOrderConsumer` and `DeliveryOrderConsumer`. The concrete observer of the first instance is a module-level function, so it adds no class.

**Unique participant symbols:** 13. One symbol is counted for each class, module-level function or module-level object mapped to a GoF role. Methods count with their class, and a symbol shared between instances counts once. Supporting rows in the mapping tables (registration, routing, callers, affected data) are not counted. The first instance contributes 3 symbols and the second 10. Three of the 13 are framework symbols: `post_save`, `InMemoryChannelLayer` and `AsyncWebsocketConsumer`.

**Rejected and uncertain candidates:** 12 rejected and 2 uncertain (Table 1). Observation scope: all Python modules of `canteenapp` at commit 83998629, scanned on 2026-10-11, with participant-level checks in the three areas named under Scope.

### 2.2 Pattern inventory

| Pattern type | Instance names | Instances | Concrete classes |
|---|---|---|---|
| Observer | Item Unavailability Cart Cleanup; Order Status Broadcast | 2 | 3 |

Canonical records: `data/patterns.csv` and `data/pattern_counts.csv`. The Strategy collaboration found in the evolved code is introduced by CH01 and is counted only in that snapshot (Section 3.1).

### 2.3 One record for each pattern instance

**Instance name:** Item Unavailability Cart Cleanup

**Pattern type and category:** Observer; GoF behavioural pattern [2], implemented through Django's signal dispatcher (framework idiom).

**Purpose and origin:** When an item's stock reaches zero, the item must disappear from every customer's cart. The code that changes stock (checkout and four item-editing views) should not need to know that carts exist. The application supplies the subject and the observer, and Django supplies the dispatch mechanism.

**Participant mapping**

| Pattern role | Actual class, interface and function | File and line range |
|---|---|---|
| Subject | `django.db.models.signals.post_save`, a `django.dispatch.Signal`: attach with `@receiver`/`connect()`, notify with `send()` | Django 6.0.4 (framework) |
| ConcreteSubject | `canteenapp.models.Item`; `Item.save()` derives `is_available` and calls `Model.save()`, which sends `post_save` | `data/code/baseline_83998629/canteenapp/models.py:123-149` (save 144–149) |
| Observer | not separately declared; Django's receiver protocol `(sender, instance, **kwargs)` | — |
| ConcreteObserver | `canteenapp.signals.remove_unavailable_item_from_cart` (function) | `.../canteenapp/signals.py:52-57` |
| Registration (supporting) | `CanteenappConfig.ready()` imports `canteenapp.signals` | `.../canteenapp/apps.py:3-8` |
| Affected data (supporting) | `canteenapp.models.Cart` | `.../canteenapp/models.py:151-168` |
| Callers (supporting) | `Order.create_order_from_cart`; `adminupdateitem`, `adminupdateitemimage`, `restaurantupdateitem`, `restaurantupdateitemimage` | `models.py:340, 355, 358`; `views.py:996, 1053, 3960, 4015` |

**Evidence:** The subject sets its own availability before saving (`models.py`):

```
144     def save(self,*args,**kwargs):
145         if self.count==0:
146             self.is_available=False
147         else:
148            self.is_available = True
149         super().save(*args,**kwargs)
```

The observer subscribes to that sender only and acts only on the unavailable state (`signals.py`):

```
52 @receiver(post_save, sender='canteenapp.Item')
53 def remove_unavailable_item_from_cart(sender, instance, **kwargs):
54     from .models import Cart
55 
56     if not instance.is_available:
57         Cart.objects.filter(item=instance).delete()
```

`CanteenappConfig.ready()` imports the signals module, which is how Django recommends connecting decorated receivers [3]. None of the five callers mentions `Cart`; each one only calls `item.save()`. Checkout contains one subtle point. Line 339 assigns an `F('count') - quantity` expression, so the comparison in `Item.save()` triggered at line 340 is not a numeric test. The method then rereads the item and saves it again (lines 352–358), and that second save is the one that announces unavailability correctly.

**Diagram:** Figure 2; source `Part1/pattern_architecture/observer_item_cart_cleanup.dot`, with the rendered PDF and PNG beside it.

**Verification:** Confirmed. Every role maps either to code or to Django's documented signal API.

The notification path runs from `Item.save()` through `post_save` to the receiver, and the subject holds no reference to its observer. The repository has no test for this path (`canteenapp/tests.py` is a placeholder), so the classification rests on the source. Two alternatives were considered. A direct call in each view would repeat the cleanup in five places, and a model method would couple `Item` to `Cart`. The mechanism has one consequence that matters for later changes: `QuerySet.update()` does not send `post_save`, so a bulk stock update would bypass the cleanup. The baseline contains no such update.

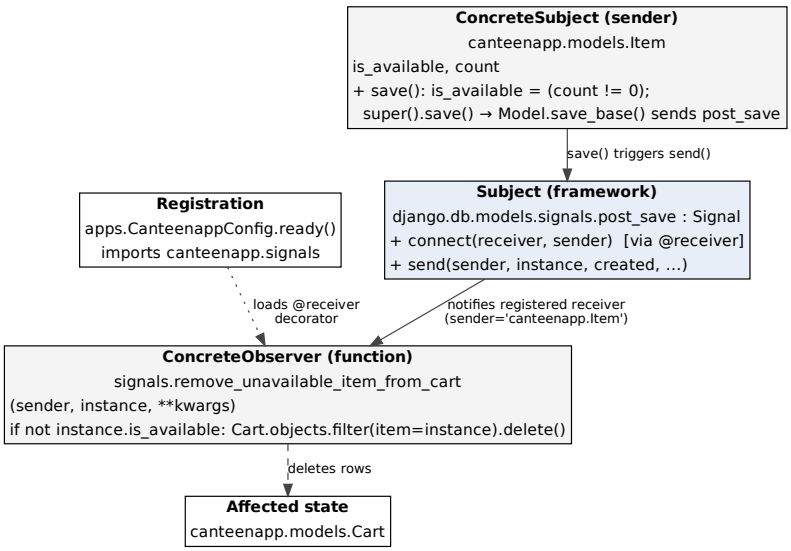



**Figure 2:** Item Unavailability Cart Cleanup: roles mapped to Django and application symbols.

**Instance name:** Order Status Broadcast

**Pattern type and category:** Observer in its publish–subscribe form; GoF behavioural pattern, mediated by Django Channels groups (framework-provided registry).

**Purpose and origin:** A status change must refresh several live dashboards (administrator, restaurant and delivery staff) without the view knowing which browsers are connected. The application supplies the notifiers and the concrete observers, and Channels handles registration and delivery.

**Participant mapping**

| Pattern role | Actual class, interface and function | File and line range |
|---|---|---|
| Subject (registry) | not declared by the application; `channels.layers.InMemoryChannelLayer` keeps group membership (`group_add`, `group_discard`, `group_send`) | `canteen/settings.py:131-135`; channels 4.3.2 |
| ConcreteSubject (notifiers) | view functions rather than a class: `adminupdateorderstatus` (representative), `deliveryupdatedeliverystatus`, `restaurantupdateorderstatus`, `restaurantplaceorder`, `phonepe_callback` | `views.py:2124-2251`, `3317-3432`, `4356-4483`, `4552-4654`, `4797-4974` |
| Observer | `channels.generic.websocket.AsyncWebsocketConsumer`; an event's `type` selects the handler method of the same name | channels 4.3.2 |
| ConcreteObserver | `AdminOrderConsumer` (group `admin_group`) | `consumers.py:350-460` |
| ConcreteObserver | `RestaurantOrderConsumer` (`restaurant_group`) | `consumers.py:462-587` |
| ConcreteObserver | `DeliveryOrderConsumer` (`delivery_group`) | `consumers.py:589-688` |
| Wiring (supporting) | `websocket_urlpatterns`; `canteen.asgi.application` | `routing.py:10-12`; `asgi.py:20-29` |

**Evidence:** Each consumer attaches itself when the socket connects and detaches when it closes (`consumers.py`):

```
351     async def connect(self):
352         user = self.scope.get('user')
353         if user and user.is_authenticated and user.role == 'admin':
354             self.group_name = 'admin_group'
355             await self.channel_layer.group_add(self.group_name, self.channel_name)
356             await self.accept()
357         else:
358             await self.close()
```

The notifier changes the order's state and then publishes events that carry only a type (`views.py`):

```
2168             order.status = status
2169             order.save()
2170             async_to_sync(channel_layer.group_send)(
2171                     'admin_group',
2172                     {
2173                         "type": 'admin_all_order'
2174                     }
2175              )
```

`adminupdateorderstatus` sends eight such events to the three groups (lines 2170–2218). Channels calls the consumer method whose name equals the event type [4]. For example, `AdminOrderConsumer.admin_all_order` (lines 365–371) re-queries the orders and sends them to the browser. The events carry no data and each observer fetches what it needs, which is the pull model described by GoF.

**Diagram:** Figure 3; source `Part1/pattern_architecture/observer_order_status_broadcast.dot`.

**Verification:** Confirmed as Observer, with one qualification. GoF lists Publish–Subscribe as another name for Observer and describes a ChangeManager that keeps the subject–observer mapping on the subject's behalf [2]; here the channel layer's groups play that part. The collaboration is not the strict form, in which a subject object stores and calls its observers. No application class holds the list, the subject role is spread over five view functions, events are dictionaries, delivery is asynchronous, and the in-memory layer works only within one process [4]. Under a strict reading it would be a framework-mediated Publish–Subscribe Channel [5], with the same participants and counts. Reading the code shows one inefficiency: every status change sends all eight events regardless of the transition, so each connected administrator runs three order queries per update.

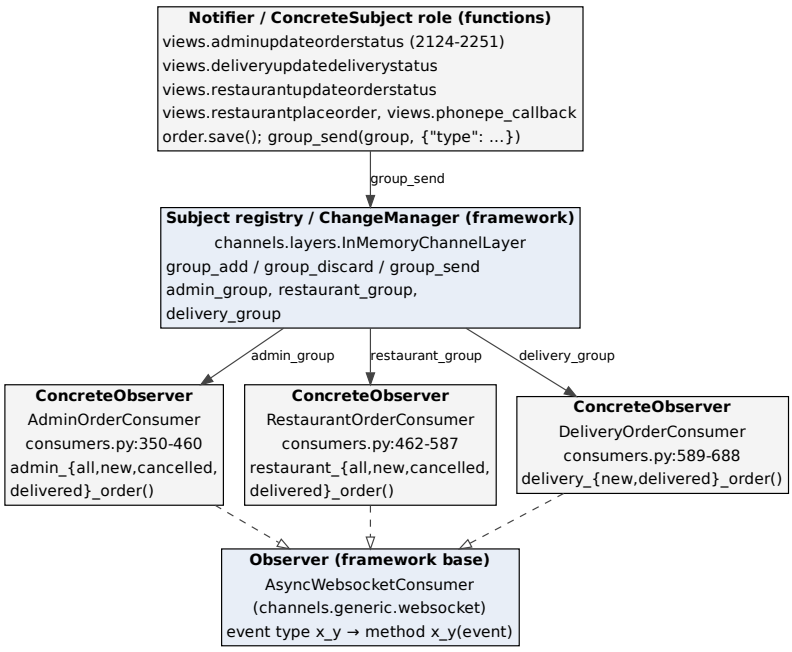



**Figure 3:** Order Status Broadcast: notifiers, framework registry and concrete observers.

**Table 1:** Rejected and uncertain candidates at the baseline (evidence: `Part1/evidence/candidate_scan.txt`).

| Candidate | Status | Reason |
|---|---|---|
| Strategy: manual delivery assignment (`views.py:1788-1821`) | rejected | One hard-coded lookup by the e-mail sent from the page; there is no family of algorithms and no interchangeable object. |
| Strategy: delivery-type branching (`Order.get_status_display`, `Order.add_additional_charges`, `get_status_index`) | rejected | `if`/`elif` on `delivery_type`; no strategy objects. |
| State: order status workflow (`models.py:260-284`, `views.py:2112-2122, 2161-2165`) | rejected | The state is a string checked against choice lists and an index; no state class carries behaviour. |

| Candidate | Status | Reason |
|---|---|---|
| Singleton: `phonepe_client` (`views.py:4701`) | rejected | A module-level object; nothing controls instantiation. |
| Facade: payment and checkout views | rejected | Long view functions coordinate several subsystems, but no class offers a simplified interface to them. |
| Adapter: PhonePe and PayU integration | rejected | Each SDK is called directly from views; there is no common gateway interface. |
| Factory: order and payment creation | rejected | Ordinary constructor and ORM `create()` calls. |
| Command: view functions | rejected | No request objects, invoker or undo. |
| Repository: data access | rejected | The ORM is used directly; there is no repository abstraction. |
| Template Method: `save()` overrides, consumer hooks | rejected | Hooks of framework base classes; the skeleton belongs to Django or Channels, not to the application. |
| Decorator: `admin_required`, `user_required` (`views.py:81-97`) | rejected | Function wrappers around `user_passes_test`; no component interface or decorator objects. |
| Observer: delivery assignment notification (`views.py:1811-1818`, group `delivery_person_<id>`) | rejected | Its only subscriber, `CustomerOrderConsumer`, sits inside a string literal (`consumers.py:691-848`) and is not routed, so no live observer exists. |
| Observer (publish–subscribe): menu, item, cart, price, search and notification groups | uncertain | Same mechanism as Order Status Broadcast across eight further groups (87 `group_send` calls in 23 view functions overall); their participants were not mapped group by group within this scope. |
| Pipeline / Chain of Responsibility: `SOCIAL_AUTH_PIPELINE` step `create_unique_username` (`settings.py:65-75`, `signals.py:7-50`) | uncertain | social-auth defines the pipeline and the application adds one step; whether a step can end the chain was not checked in the library source. |

# 3 Part 2 Evolution and refactoring

**Change and acceptance criteria: CH01, interchangeable delivery-assignment policies.** An administrator assigning a confirmed delivery order should be able to choose how the delivery person is picked, and a new rule should not require rewriting the assignment view. Acceptance criteria, each covered by tests in the CH01 package:

1. the existing manual payload `{"order_id", "delivery_personnel"}` still assigns the chosen person, sets the status to `shipped` and returns `success: true`;
2. the payload `{"order_id", "policy": "least_workload"}` assigns, without an e-mail, the eligible person with the fewest unfinished delivery orders, breaking ties by the lowest user ID;
3. only active, non-superuser accounts with the role `delivery_person` and the `canteenapp.is_deliveryperson` permission, held directly or through a group, are eligible;
4. only orders still in the assignment queue (delivery type, confirmed, confirmed by the customer, unassigned) can be assigned;
5. malformed input returns 400, a missing order 404, a stale order or an empty candidate set 409, and a database failure 503;

6. the admin page offers both policies with manual as the default, the existing channel event is still sent, and no schema change is needed.

**Before and after:** Before: commit `83998629`. After: the working tree `data/code/evolved_CH01/` and the unified diff `Part2/changes/Change1/CH01.diff`; CH01 was not committed to Git. Applying the diff to a fresh copy of the baseline reproduces the four changed files byte for byte (`Part2/changes/Change1/rerun_2026-10-11/RECHECKS.md`). Changed paths:

- `canteenapp/delivery_assignment.py` (added);
- `canteenapp/views.py` (modified);
- `canteenapp/templates/adminassignorder.html` (modified);
- `canteenapp/test_delivery_assignment.py` (added).

Models, migrations, settings, consumers, signals, routes and `requirements.txt` are unchanged (`data/provenance/changed-files.json`).

**Implementation and tests:** The new module declares the policy abstraction:

```
36 class DeliveryAssignmentPolicy(ABC):
37     @abstractmethod
38     def select(self, candidates, *, email=None):
39         """Return one eligible DeliveryPerson, or raise AssignmentError."""
```

`ManualAssignmentPolicy` (lines 42–49) returns the eligible candidate with the supplied e-mail. `LeastWorkloadAssignmentPolicy` (lines 52–66) annotates each candidate with a distinct count of assigned delivery orders that are neither `delivered` nor `cancelled`, then orders by that count and by primary key. An unknown status value therefore counts as unfinished work, which is the cautious choice. `DeliveryAssignmentService` is the context. Inside one transaction it checks that the order is still assignable, delegates the choice and writes through a conditional update that repeats the eligibility filter:

```
101         if not assignable.exists():
102             raise AssignmentError("Order is no longer eligible for assignment.", 409)
103         person = self.policy.select(eligible_delivery_people(), email=email)
104         updated_at = timezone.now()
105         # Guard the write as well: SQLite does not implement SELECT FOR UPDATE.
106         if assignable.update(delivery_person=person, status="shipped", updated_at=updated_at) !=
      1:
107             raise AssignmentError("Order changed; reload before assigning again.", 409)
```

The view now:

- validates the JSON body;
- looks up the policy by name with `get_assignment_policy(policy_name)`, defaulting to `manual` (`views.py:1815`);
- maps each `AssignmentError` to its HTTP status;
- schedules the existing `delivery_person_<id>/new_order` event with `transaction.on_commit(..., robust=True)`, so a notification failure is logged rather than reported as a failed assignment.

The page gained a policy selector and disables the manual drop-down in automatic mode. It now submits the order selected in `ordersData[currentOrderIndex]` instead of reading a duplicated `orderid` element, ignores repeated clicks and shows server messages. A CSRF test exposed a missing cookie on the assignment page, so `adminassignorder` now sets it.

Commands and results (working directory `CH01/working`; logs in `Part2/changes/Change1/evidence/`):

| Command | Result | Log |
|---|---|---|
| `python manage.py test canteenapp.test_delivery_assignment -v 2` | 39 tests, OK (0.294 s); exit 0 | `tests-default-final.log` |
| `node verification/test_assignment_ui.cjs` | 9 passed; exit 0 | `ui-tests.log` |
| `python manage.py check` | exit 0; 1 warning (`ACCOUNT_EMAIL_REQUIRED`) | `check-default-final.log` |
| `python manage.py makemigrations --check --dry-run` | No changes detected; exit 0 | `migrations-check.log` |
| `python -m pip check` | exit 1; six conflicts among baseline pins | `pip-check-final.log` |

The 39 Django tests are 18 policy tests and 21 endpoint tests. They run with the normal `canteen.settings`, real migrations in an isolated test database, the real admin decorator and CSRF middleware, and the in-memory channel layer. Their coverage includes:

- delegation to an injected third policy;
- unequal loads, ties at zero and non-zero load, and every non-terminal status;
- exclusion of terminal, pickup and dining work;
- each eligibility rule, group permissions and duplicate permission paths;
- invalid and missing input, stale and terminal orders, and repeated assignment;
- authorisation, CSRF rejection and a CSRF-token round trip;
- removal from the assignment queue, and notification and database failures.

The nine JavaScript tests run the template's script under Node with stubbed DOM and `fetch`. The logs also keep the earlier failed runs. On 2026-10-11 the UI tests and the patch check were re-run with the same results (`rerun_2026-10-11/`).

**Pattern effects:** Delivery Assignment Policy Selection, Strategy: `newly_introduced`. It replaces the hard-coded lookup that was rejected as a Strategy candidate at the baseline rather than extending a confirmed instance. Item Unavailability Cart Cleanup and Order Status Broadcast, Observer: `preserved`. Their files are unchanged; only their line numbers in `views.py` moved, by 13 lines for the notifiers and 6 for the item-update callers. Participant mapping after CH01:

| Pattern role | Actual class and function | File and line range (`data/code/evolved_CH01/`) |
|---|---|---|
| Strategy | `DeliveryAssignmentPolicy` (ABC, `select`) | `canteenapp/delivery_assignment.py:36-39` |
| ConcreteStrategy | `ManualAssignmentPolicy` | `delivery_assignment.py:42-49` |
| ConcreteStrategy | `LeastWorkloadAssignmentPolicy` | `delivery_assignment.py:52-66` |
| Context | `DeliveryAssignmentService` (`assign`) | `delivery_assignment.py:81-111` |
| Client (supporting) | `views.adminassignordertodelivery` | `canteenapp/views.py:1803-1834` |
| Registry (supporting) | `POLICIES`, `get_assignment_policy` | `delivery_assignment.py:69-78` |
| Candidate query (supporting) | `eligible_delivery_people` | `delivery_assignment.py:18-33` |

**Refactoring:** The baseline view (`views.py:1788-1821`) showed several code smells:

- one function parsed the request, looked up the person and the order, changed the order, printed

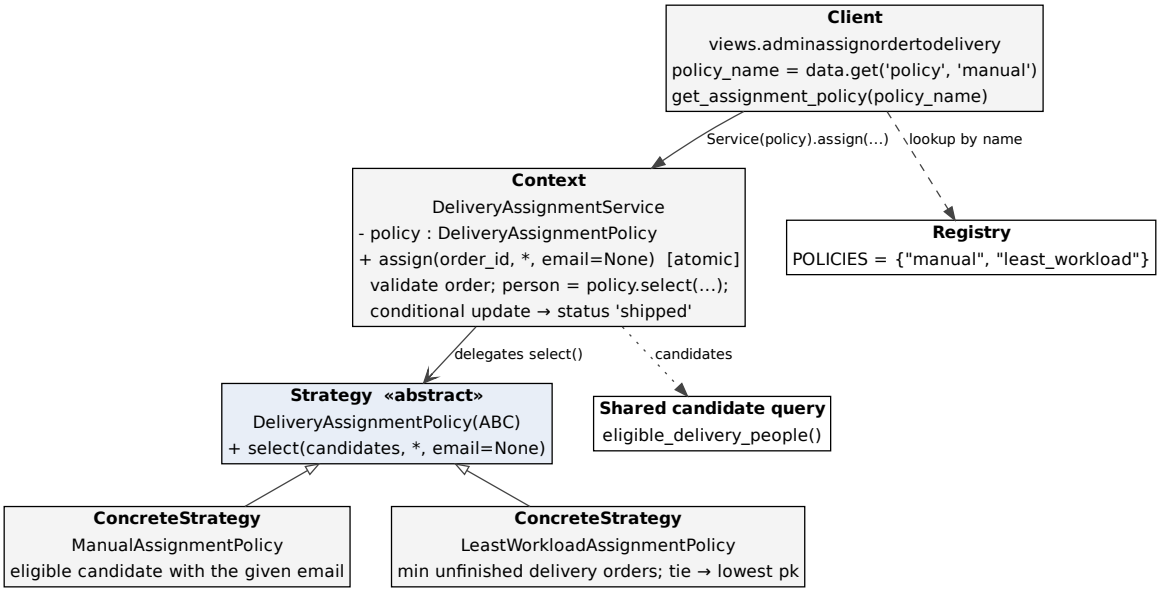



**Figure 4:** Delivery Assignment Policy Selection after CH01. Source: `Part2/diagrams/strategy_delivery_assignment_CH01.dot`.

debug output and published the event;
- `json.loads` had no error handling, so a malformed body caused a server error;
- a catch-all `except Exception` returned HTTP 200 with `success: false`;
- nothing stopped a delivered, cancelled or already-assigned order from being reassigned.

CH01 applied Extract Class (selection and persistence moved to `DeliveryAssignmentService`) and Extract Function (`eligible_delivery_people`, `_notify_delivery_assignment`) [6], and it replaced the single hard-coded rule with polymorphic policy objects. Three tests show that the existing behaviour survives:

- `test_legacy_manual_payload_and_existing_event` sends the old payload and checks the response, the assigned person, the `shipped` status and the original group and event type;
- `test_actual_inmemory_channel_receives_preserved_event` receives that event on the real in-memory layer;
- `test_queue_to_assignment_flow_removes_assigned_order` confirms that the order leaves the assignment queue.

Some behaviour changed deliberately: invalid requests now receive 4xx/5xx codes, orders outside the queue are refused, and the write uses `QuerySet.update()`. That last choice bypasses `Order.save()` and its `clean()` call, so a future `post_save` receiver for `Order` would not see assignments; none exists today.

**Before/after metrics:** Both snapshots were measured with the same tool and rules: lizard 1.24.1, counting non-blank, non-comment lines (NLOC) and cyclomatic complexity (CCN) [8, 7]. Script: `Part2/changes/Change1/metrics/measure_ch01.py`; output: `ch01_metrics.csv`.

| Measure | Baseline | After CH01 |
|---|---|---|
| `views.py` NLOC | 4,651 | 4,660 |
| `adminassignordertodelivery` NLOC / CCN | 30 / 5 | 27 / 5 |
| `delivery_assignment.py` NLOC / maximum function CCN | — | 77 / 6 (`assign`) |
| Test code NLOC (`test_delivery_assignment.py`) | 0 | 300 |
| Diff lines added/removed: module, view, template, tests | 111/0, 41/28, 52/27, 353/0 | |
| Application tests passing | no application tests | 39 Django + 9 JS |

The template is measured by diff lines only, because lizard does not parse Django templates.

**Change and acceptance criteria: CH02 and CH03.** The team's files contain no requirement statement or acceptance criteria for these changes. The earlier proposals, `data/legacy_datasets/change_scenarios.csv` and `Part2/llm/historical_note_evolution_proposals_20260912-064608.md`, list three candidates: notifying new recipients of order-status changes through the Observer collaboration, handling cancellation and refunds as explicit states, and restaurant availability changes.

**Before and after:** No commit, diff or changed file exists for either change.

**Implementation and tests:** Neither change was implemented, so no test was written or run.

**Pattern effects:** None. The counts below have no CH02 or CH03 snapshot.

**Refactoring:** None.

**Before/after metrics:** Not applicable.

### 3.1 Updated pattern counts

| Snapshot | Types | Instances | Concrete classes | Rejected / uncertain |
|---|---|---|---|---|
| Baseline (83998629) | 1 | 2 | 3 | 12 / 2 |
| After CH01 | 2 | 3 | 5 | 11 / 2 |

After CH01 there are two Observer instances, both preserved under their original names, and one newly introduced Strategy instance. The concrete classes gained are `ManualAssignmentPolicy` and `LeastWorkloadAssignmentPolicy`, and unique participant symbols rise from 13 to 17. The baseline rejection of manual delivery assignment as a Strategy no longer applies because that code was replaced, which leaves 11 rejected candidates. Records: `data/pattern_evolution.csv` and `data/pattern_counts.csv`.

**Interpretation:** CH01's main benefit is a clear place for new assignment rules: a third policy needs one class and one registry entry, while order checks, the transaction and the notification stay in one place. A test demonstrates this by injecting a policy that the registry does not know. The costs are an extra module to follow, a name-based registry that is checked only at run time, and a context with complexity 6 that now holds state checks the view never had. Production NLOC grew by 86 (77 in the new module and 9 in the view), a figure that says nothing about design quality on its own. "Least workload" counts unfinished orders. It does not measure availability, location or shift, because the model has no such fields.

The remaining limits lie outside CH01:

- `pip check` reports six conflicts among the baseline pins, and CH01 was validated in an environment installed with `--no-deps`;
- Firebase credentials are absent, and browser rendering was not tested;

- the assignment event goes to a group that no routed consumer joins;
- SQLite ignores `select_for_update`, so concurrent assignments rely on the conditional write and may return 503.

The tests establish that the feature behaves as specified, not that the application is ready to deploy.

# 4 Part 3 Cross-language comparison

**Selected instances:** For Observer, *Order Status Broadcast*: the demonstrations' `Admin`, `Restaurant` and `DeliveryPerson` observers correspond to the three consumer groups, and `Order.setStatus` plays the role of the notifying view. For Strategy, *Delivery Assignment Policy Selection*, the only confirmed Strategy. The demonstrations predate CH01 and implement the earlier delivery-partner proposal (scenario CS1). Their `AvailabilityStrategy` and `PriorityStrategy` use `available` and `priority` attributes that the application's `DeliveryPerson` lacks, so they model the pattern structure rather than CH01's two policies. Both programs are simplified models, not translations of the Django code. They are kept unchanged, and their SHA-256 hashes match `data/provenance/demonstrations-before.json`.

**Common specifications:** The shared fixture is in `Part3/test-cases/cases.json`.

- **Observer.** Order `0101` moves to `Shipped` with three observers registered in this order: Admin, Restaurant and `DeliveryPerson("Delivery Partner")`. Each observer prints one line, in registration order.
- **Strategy.** Partner A is available with priority 2, B is unavailable with priority 5, and C is available with priority 4. The availability rule returns the first available partner (A). The priority rule returns the highest-priority available partner (C), so B is skipped.

The expected standard output is in `Part3/test-cases/expected_output.txt`. A defined error case is not reachable from the hard-coded fixture: when no partner is available, each language returns its null value and prints "No delivery partner available".

**Implementations:** Each language has one file under `Part3/order_events_and_delivery_selection/` that contains both patterns:

- `java/DesignPatterns.java` (OpenJDK 21.0.12.1);
- `python/design_patterns.py` (CPython 3.13.16);
- `javascript/design-patterns.js` (Node.js 22.22.0);
- `cpp/design_patterns.cpp` (g++ 13.3.0, default flags as in the team's recorded command).

Each folder has a `README.md` and a `run_tests.sh`. The script builds into a temporary directory where needed, runs the program and passes its output to the shared checker `Part3/tools/check_output.py`.

**Actual results:** All four implementations pass every case: Observer 3/3, Strategy 2/2 and whole-output 1/1, each with exit status 0. Logs: `Part3/evidence/test_java.log`, `test_python.log`, `test_javascript.log` and `test_cpp.log`. The earlier table `data/legacy_datasets/four_language_execution.csv` records the same outputs without logs.

**Static and dynamic comparison:** Markers in `Part3/tools/measure.py` split each file into Observer lines, Strategy lines and shared scaffolding (imports, the `main` signature, closing braces); the split is saved in `Part3/evidence/line_partition.json`. LOC counts non-blank, non-comment lines. CC is lizard's maximum per-function CCN over functions that start in each part (`Part3/evidence/static_metrics.csv`, `lizard_functions.csv`).

| Language | Runtime / compiler | Observer LOC | Observer max CC | Strategy LOC | Strategy max CC | Shared LOC | File NLOC |
|---|---|---|---|---|---|---|---|
| Java | OpenJDK 21.0.12.1 | 45 | 2 | 60 | 5 | 6 | 111 |
| Python | CPython 3.13.16 | 31 | 2 | 39 | 4 | 0 | 70 |
| JavaScript | Node.js 22.22.0 | 44 | 1 | 49 | 3 | 0 | 93 |
| C++ | g++ 13.3.0 | 50 | 2 | 62 | 5 | 7 | 119 |

Timing followed the faculty procedure: compile first, run once to warm up, then take five measured runs on the same machine (Linux 6.18, x86_64 cloud container) on 2026-10-11. Each figure is the wall-clock time of the whole process. It includes runtime start-up, both pattern sections and output capture, and it excludes compilation. Every measured run produced the expected output (`Part3/evidence/timing_raw.csv`).

| Combined Observer + Strategy process | Median (ms) | Min (ms) | Max (ms) |
|---|---|---|---|
| Java (`java -cp <build> DesignPatterns`) | 70.7 | 65.2 | 71.9 |
| Python (`python3 design_patterns.py`) | 18.9 | 15.9 | 20.4 |
| JavaScript (`node design-patterns.js`) | 34.7 | 32.8 | 40.6 |
| C++ (native binary) | 1.9 | 1.8 | 1.9 |

Each program runs both patterns in one process, so the timings are combined figures. For that reason the per-pattern `median_process_ms` cells in `data/cross_language.csv` are left blank. With so little work per run, the differences reflect the start-up cost of the JVM, Node and CPython rather than the cost of either pattern.

**Comparison:** All four programs have the same participants:

- an Observer type with three concrete observers, and a subject (`Order`) that stores a list and iterates over it;
- a Strategy type with two concrete strategies, and a context that receives its strategy through the constructor.

The languages differ first in how strictly the abstraction is declared. Java uses an `interface` and C++ an abstract class with pure virtual functions and a virtual destructor, and both compilers enforce them. Python and JavaScript use ordinary base classes with empty methods, so a missing `update` surfaces only when it is called. Python's `abc` module, which CH01 uses, would make the contract explicit [9]. They also differ in ownership. The C++ subject stores non-owning raw pointers to stack objects, and `assign` returns a pointer into the caller's vector that is valid only while that vector exists; the other three languages use garbage-collected references. Complexity is highest in the Java and C++ `PriorityStrategy.assign`, at CC 5 (the loop, the `if` and two boolean operators). Python's list comprehension with `max(key=...)` gives 4, and JavaScript's `filter`/`reduce` with separate arrow functions gives 3.

**Technology discussion:** In the application, Observer is never hand-written; it comes from two framework mechanisms with different guarantees. Django signals run synchronously inside `save()`, in the same process and transaction, and are connected when the app loads [3]. Channels groups deliver asynchronously over WebSocket connections and dispatch on the event type. The in-memory layer is intended for development and testing, not for multi-process deployment [4]. CH01, by contrast, writes Strategy explicitly with `abc.ABC`, because no framework hook offers interchangeable selection rules.

# 5 LLM records and human verification

**Run and model:**

| Part | Displayed model and date | Record |
|---|---|---|
| 1a repository selection | not recorded; a link in the note carries `utm_source=chatgpt.com` | pasted output only |
| 1b pattern analysis | not recorded; same indication | pasted output only |
| 2 change proposals | not recorded; the file name holds the paste time 2026-09-12 06:46 | pasted output only |
| Report production | Claude Opus 5.5 in the Claude app (identifier `claude-opus-5-5`), 2026-10-11; Claude Pro plan | two exact prompts saved |

Canonical records: `data/llm.csv`. For the three historical analyses only pasted outputs survive, and they are recorded as such.

**Inputs and outputs:** Historical outputs are in `Part1/llm/historical_note_repository_selection.md`, `Part1/llm/historical_note_pattern_analysis.md` and `Part2/llm/historical_note_evolution_proposals_20260912-064608.md`; the inputs given to those sessions were not recorded. For report production, the exact prompts are `data/llm/2026-10-11_report_generation_prompt.md` and `data/llm/2026-10-11_report_revision_prompt.md`. The supplied files, with their hashes and the source commit 83998629, are listed in `data/llm/2026-10-11_report_generation_inputs.md`. The outputs are this report, the CSV datasets, the README, the diagrams and the measurement scripts.

**Claims and evaluation:**

- **Part 1b note: partially correct.** It rightly identifies the Channels broadcast as Observer-like and rejects State, Strategy, Facade, Singleton, Adapter, Factory, Command, Repository and Template Method. It misses the `post_save` collaboration, does not separate the publish–subscribe form from strict Observer, and overlooks that the subscriber of the assignment event is disabled inside a string literal (`Part1/evidence/candidate_scan.txt`).
- **Part 2 note: partially correct, as a set of proposals.** Its Strategy proposal was implemented in CH01 with different policies, because the model has no availability or priority fields. Its other proposals remain unimplemented.
- **Part 1a note: cannot fully confirm.** Its description of the chosen repository matches the code, but its claims about the other repositories were not re-checked.
- **Report production: requires team review.** Each figure in this report was traced to a log or a script output.

**Generated code:** The team states that no AI tool was used to write the CH01 code or the four demonstrations. The report-production session wrote the Part 3 test runners and checker, `Part3/tools/measure.py`, `Part1/evidence/scan_candidates.sh`, `Part2/changes/Change1/metrics/measure_ch01.py`, `data/code/verify_baseline.py` and the Graphviz diagram sources. Each was executed, and its output is stored beside it. None of them modifies the application or the demonstrations.

# 6 Findings, contributions and references

**Findings:**

1. At commit 83998629 the application contains one confirmed pattern, Observer, in two instances. Both rely on framework mechanisms rather than hand-written subjects: Django's `post_save`

signal for cart cleanup and Channels groups for live order updates.
2. The subject role in the order broadcast is spread over five view functions that publish to the same groups. New observers are easy to add, but the sending side is repetitive: `adminupdateorderstatus` sends eight events for every status change.
3. The delivery-assignment notification has no live subscriber at the baseline, because its only consumer is disabled inside a string literal and is not routed. CH01 preserves the event as it was.
4. CH01 introduces a Strategy with a clear abstraction, two policies and a context that owns validation and persistence. 39 Django tests and 9 script tests pass in the recorded validation environment.
5. The four-language demonstrations produce identical results on the shared fixture. Their timings mainly reflect runtime start-up.

**Contributions:**

| Roll number | Name | Batch | Contribution | Source and evidence |
|---|---|---|---|---|
| AM.SC.U4CSE23001 | Aadil Nizar | CSE A | Refactoring and CH01 implementation | `Part2/changes/Change1/`, `data/code/evolved_CH01/` |
| AM.SC.U4CSE23007 | Adil Najeem | CSE A | Baseline analysis | `data/code/baseline_83998629/`, `data/provenance/`, Section 1 |
| AM.SC.U4CSE23014 | Athul Yesudas | CSE A | Pattern discovery | `Part1/`, `data/patterns.csv` |
| AM.SC.U4CSE23039 | Hima C | CSE A | Testing and validation | `Part2/changes/Change1/evidence/`, `Part3/evidence/` |
| AM.SC.U4CSE23046 | M Krishnapriya | CSE A | Cross-language comparison | `Part3/`, `data/cross_language.csv` |
| AM.SC.U4CSE23053 | Muhammed Faais A | CSE A | Report and repository preparation | `A1_Report.tex`, `README.md`, `data/` |

**References:**


[1] H. Vemali, *campus-food-ordering* (Canteen Management System), GitHub, commit 83998629e633594037cfa25e583542f9f6f0f968, MIT licence. `https://github.com/HarshavardhanVemali/campus-food-ordering`

[2] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994 (Observer, pp. 293–303; Strategy, pp. 315–323).

[3] Django Software Foundation, "Signals", Django 6.0 documentation. `https://docs.djangoproject.com/en/6.0/topics/signals/` (accessed 2026-10-11).

[4] Django Channels, "Channel Layers", Channels documentation. `https://channels.readthedocs.io/en/latest/topics/channel_layers.html` (accessed 2026-10-11).

[5] G. Hohpe and B. Woolf, *Enterprise Integration Patterns*. Addison-Wesley, 2003 (Publish-Subscribe Channel).

[6] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018 (Extract Class, Extract Function).

[7] T. J. McCabe, "A complexity measure", *IEEE Transactions on Software Engineering*, SE-2(4):308–320, 1976.

[8] T. Yin, *lizard* code complexity analyser, version 1.24.1. `https://github.com/terryyin/lizard`

[9] Python Software Foundation, "abc — Abstract Base Classes", Python 3 documentation. `https://docs.python.org/3/library/abc.html`

# 7 Team folder and datasets (take a screenshots from the GitHub repository— your team)

Figure 5 shows the team folder on GitHub after the first upload (commit `82a5d58`, "A1 SUBMISSION"). The folder contains the report, its LaTeX source, the README and the four folders below; the screenshot image is stored at `data/test_result/github_team_folder_82a5d58.png`.

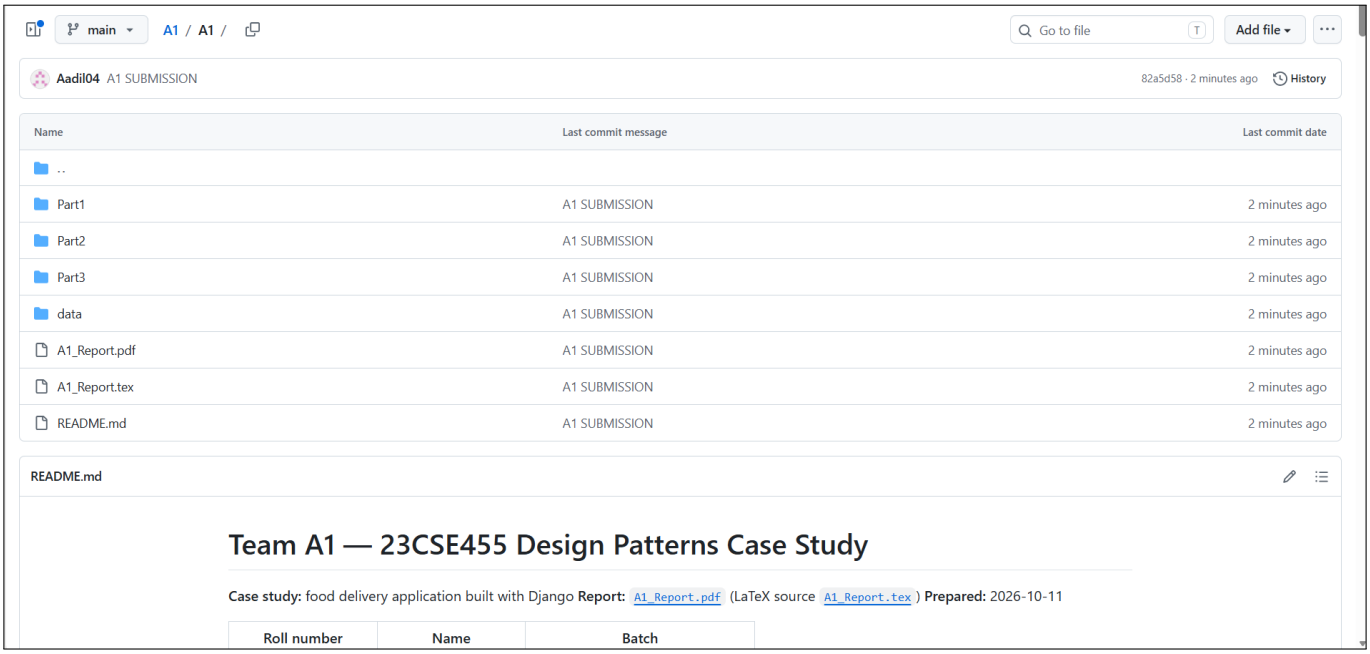



**Figure 5:** GitHub listing of the team folder (breadcrumb "A1 / A1", branch `main`) after commit `82a5d58`, "A1 SUBMISSION" by Aadil04. It shows `Part1/`, `Part2/`, `Part3/`, `data/`, `A1_Report.pdf`, `A1_Report.tex` and `README.md`, with the start of the rendered README below.

```
A1/
  README.md, A1_Report.tex, A1_Report.pdf
  data/   patterns.csv, pattern_counts.csv, pattern_evolution.csv,
          cross_language.csv, llm.csv
          code/baseline_83998629/   original application
          code/evolved_CH01/        application after CH01
          provenance/, legacy_datasets/, llm/, test_result/
  Part1/  pattern_architecture/, evidence/, llm/
  Part2/  changes/Change1/ (requirement, diff, source, tests, logs, metrics),
          changes/Change2/, changes/Change3/, diagrams/, llm/
  Part3/  order_events_and_delivery_selection/{java,python,javascript,cpp}/,
          test-cases/, tools/, evidence/, llm/
```

The faculty documents give some datasets more than one name (`Pattern_count.csv`, `patterns_evolution.csv`, `pattern_evalution.csv`, `pattern_crosslanguage.csv`) and ask that the CSVs not be repeated inside the part folders. One canonical copy of each is therefore kept in `data/`, with the column names exactly as given in the instructions; the name mapping is recorded in `README.md`.